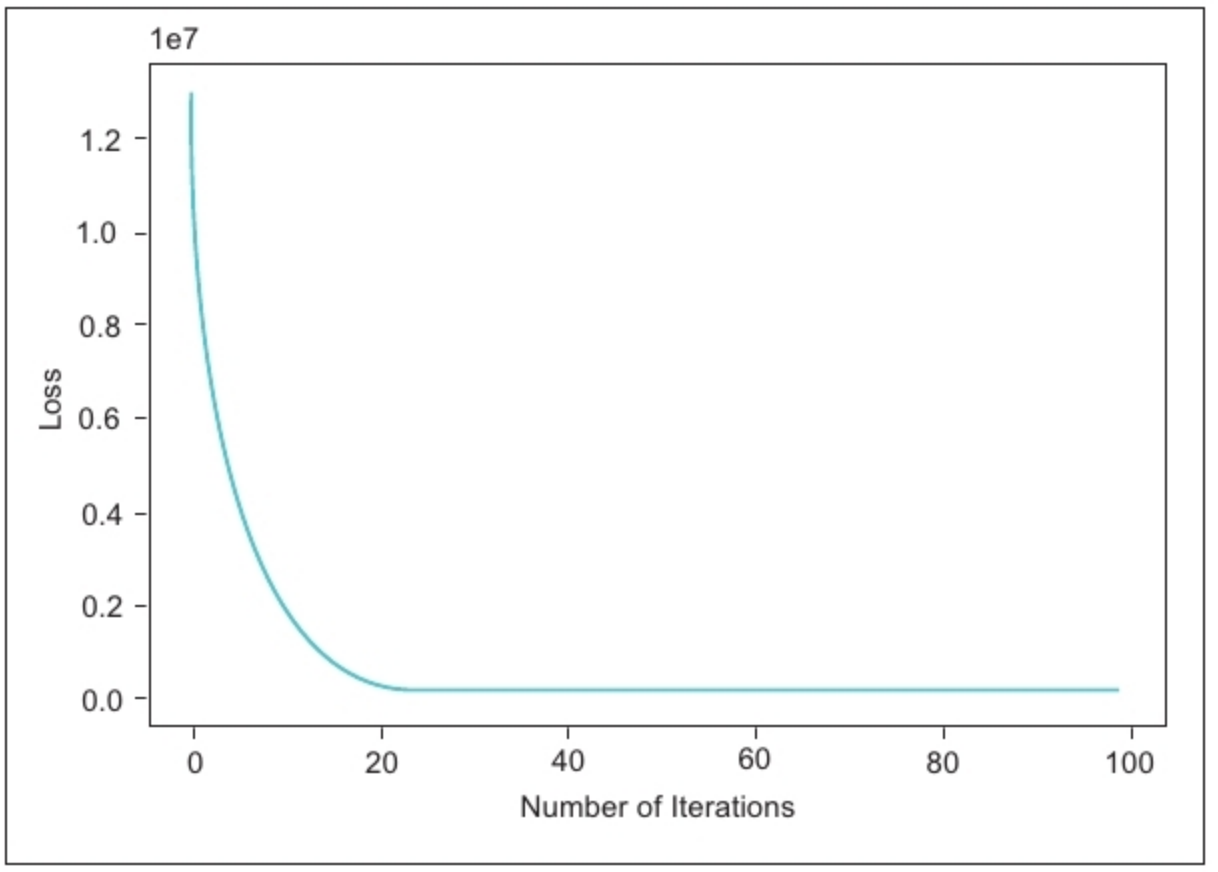

Figure 5: Visualisation of the loss rate

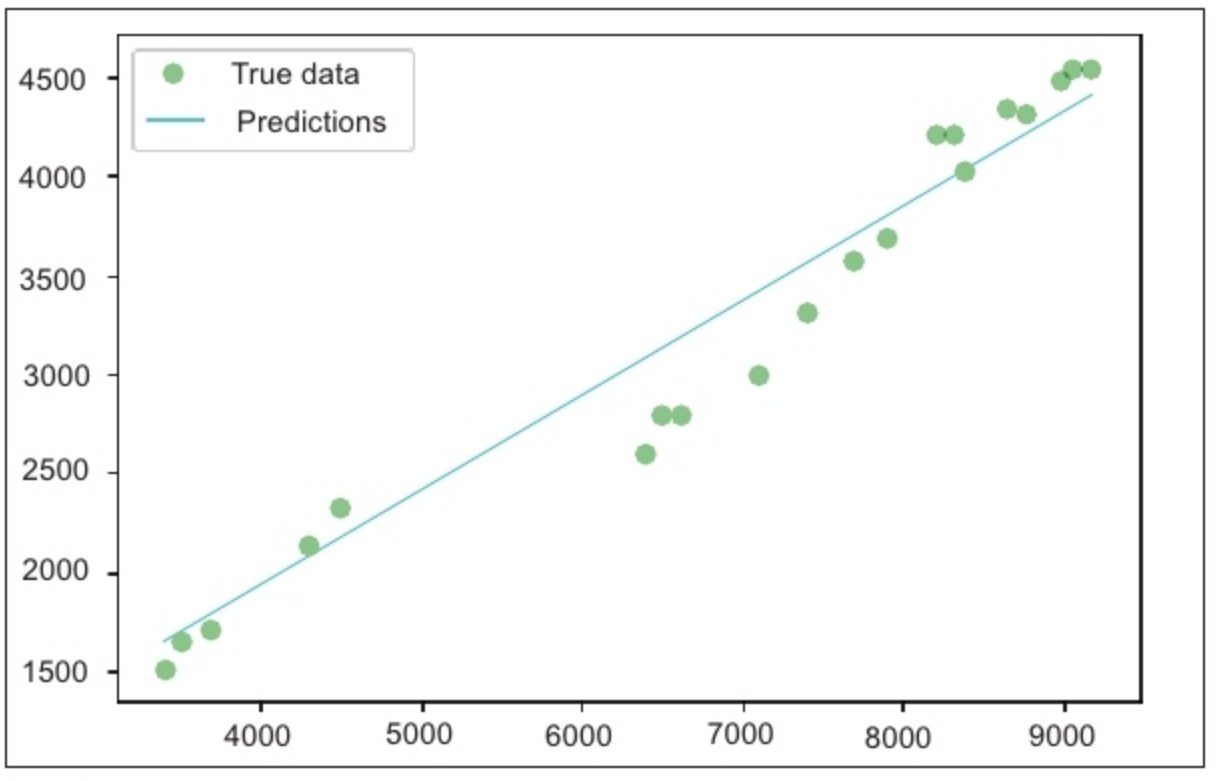

Figure 6: Predicting the values of training data

Next, we will initialise the loss (Mean Squared Error) and optimisation (Stochastic Gradient Descent) functions that we'll use in the training of this model:

```
criterion = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.
parameters(), lr=learn_rate)
```

After completing all the initialisations, we can now begin to train our model. The following code is for training the model:

```
losses=[]
for epoch in range(epochs):
    # Convert the inputs and labels to
Variables
    if torch.cuda.is_available():
        inputs = Variable(torch.from_
numpy(x_train).cuda())
        labels = Variable(torch.from_
numpy(y_train).cuda())
    else:
        inputs = Variable(torch.from_
numpy(x_train))
        labels = Variable(torch.from_
numpy(y_train))

    # Clear gradient buffers in order
to avoid cumulation of gradients
    optimizer.zero_grad()

    # get output from the model
    outputs = model(inputs)

    # get the loss for the predicted
output
    loss = criterion(outputs, labels)
    # get the gradients
    loss.backward()
    # update parameters
    optimizer.step()
    losses.append(loss.data)
    if(epoch%10==0):
        print('epoch {}, loss {}'.
format(epoch, loss.data))
plt.plot(range(epochs),losses)
plt.xlabel("Number of Iterations")
plt.ylabel("Loss")
plt
```

The above code will give you a visualisation of the performance of the loss function over 100 epochs. Figure 5 shows that the loss rate is steadily decreasing and is at its lowest near the 100th iteration.

Now that our linear regression model is trained, let's test it. We will first use our model to predict the values of the training data itself, and visualise the line to see how accurately we are able to predict the values.

```
with torch.no_grad():
    if torch.cuda.is_available():
        predicted =
model(Variable(torch.from_numpy(x_
train).cuda())).cpu().data.numpy()
    else:
        predicted =
model(Variable(torch.from_numpy(x_
train))).data.numpy()
    print("predicted=",predicted)

plt.clf()
```